

2026 PORTFOLIO



Siho Kim

Linux Thermal Framework 분석

DTS 기반 Thermal Zone 생성부터

CPUFreq Cooling Device를 통한 CPU Throttling까지 전체 동작 흐름 분석

<https://github.com/KimSiHo/jetson-bsp>

CONTENTS

1	개요	<u>4</u>
2	프로젝트 구조	<u>5</u>
3	기술 배경지식	<u>6</u>
4	테스트	<u>11</u>
5	결과 해석	<u>14</u>

목표

- DTS의 thermal-zone 설정부터 CPUFreq Cooling Device를 통한 CPU 주파수 제한까지

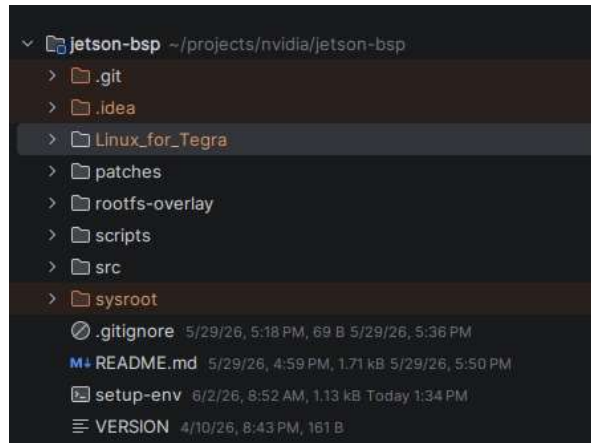
Linux Thermal Framework의 전체 동작 흐름 분석

- Thermal Zone, Trip Point, Thermal Governor(step wise), Cooling Device 간의 동작 흐름 분석

- sysfs의 emul_temp를 이용하여 온도 이벤트를 발생시키고 CPUFreq Cooling Device에 의한 실제 주파수 제한 적용 확인

프로젝트 구조

- 기본 BSP는 NVIDIA L4T를 사용하며 커스텀 수정 사항은 patches와 rootfs-overlay로 관리
- 배포 대상 애플리케이션 소스는 src에서 관리하며 크로스 빌드를 위한 sysroot 환경 구성
- scripts에는 sysroot 생성, 패치 적용, 이미지 생성 자동화 스크립트를 포함
- Yocto 대비 특징
 - 환경 구축이 빠르고 단순함
 - NVIDIA BSP를 즉시 활용 가능
 - Yocto는 양산 환경에 적합한 전체 시스템 빌드 환경 제공



전체 흐름

dts

- Thermal Zone, Trip Point, Cooling Map을 정의하여 Thermal 정책을 구성

bpmp

- Thermal Sensor를 관리하고 Trip Point 초과 시 Linux Thermal Framework에 이벤트 전달

thermal

- 온도 갱신 및 Trip Point를 검사하고 Thermal Governor를 통해 Cooling Device를 제어

throttling

- Cooling Device가 CPUFreq 정책을 제한하여 CPU 주파수를 낮추고 발열을 제어

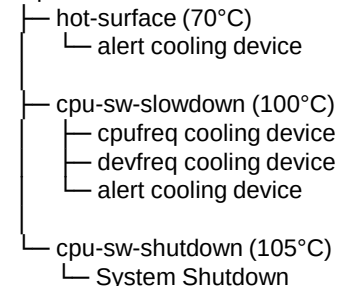
기술 배경지식

dts

- 부팅 과정에서 thermal-zones 노드가 파싱되어 thermal zone로 등록됨
- BPMP Thermal Driver probe시 cpu-thermal zone에 센서가 연결됨
- cooling-map은 특정 trip point와 cooling device를 매핑하여 온도 초과 시 수행할 냉각 동작을 정의

- 정리하면 다음과 같음

Trip Point



- cpufreq cooling device는 CPU의 동작 주파수를 제한하여 발열을 제어하는 Cooling Device
- devfreq cooling device는 장치의 동작 주파수를 제한하여 발열을 제어하는 Cooling Device
- alert cooling device는 sysfs를 통해 온도 초과 상황을 사용자 공간에 전달하기 위한 메커니즘을 제공하는 Cooling Device

```
cpu-thermal {
    thermal-sensors = <0x105 0x88>;
    status = "okay";
    polling-delay = <0x3e8>;
    polling-delay-passive = <0x3e8>;

    trips {

        cpu-sw-slowdown {
            temperature = <0x102b8>;
            hysteresis = <0x00>;
            type = "passive";
            phandle = <0x106>;
        };

        cpu-sw-shutdown {
            temperature = <0x10034>;
            hysteresis = <0x00>;
            type = "critical";
            phandle = <0x313>;
        };

        hot-surface {
            temperature = <0x11178>;
            hysteresis = <0x1f40>;
            type = "passive";
            phandle = <0x109>;
        };
    };

    cooling-maps {

        map-cpufreq {
            cooling-device = <0x134 0xffffffff 0xffffffff 0x136 0xffffffff 0xffffffff 0x13a 0xffffffff 0xffffffff>;
            trip = <0x106>;
        };

        map-devfreq {
            trip = <0x106>;
            cooling-device = <0x1d7 0xffffffff 0xffffffff>;
        };

        map-throttle-alert {
            trip = <0x106>;
            cooling-device = <0x1d8 0x01 0x01>;
        };

        map-hot-surface-alert {
            trip = <0x109>;
            cooling-device = <0x1da 0x01 0x01>;
        };
    };
};
```

bpmp

- Tegra SoC에서는 BPMP Firmware가 Thermal Sensor 및 Trip Point Monitoring을 담당

```
err = tegra_bpmp_request_mrqs(bpmp, MRQ_THERMAL, bpmp_mrqs_thermal,  
                             tegra);
```

- Trip Point 초과 시 BPMP가 해당 Thermal Zone 정보를 Linux에 전달
- 워크 핸들러에서 thermal_zone_device_update 를 호출함으로써 호출 제어가 thermal core 쪽으로 넘어감

```
for (i = 0; i < tegra->num_zones; ++i) {  
    if (tegra->zones[i]->idx != req->best_trip_reached_zone)  
        continue;  
  
    schedule_work(&tegra->zones[i]->tz_device_update_work);  
    tegra_bpmp_mrqs_return(ch, 0, NULL, 0);  
    return;  
}
```

- BPMP 와 Linux의 역할 분담은 다음과 같음

BPMP 역할

- Thermal Sensor 데이터 제공
- Trip Point Monitoring
- 온도 이벤트 발생 시 Linux 통보

Linux 역할

- Thermal Framework 처리
- Thermal Governor 판단
- Cooling Device 제어

thermal

- 워크 핸들러(thermal_zone_device_update)에서 Thermal Zone의 현재 온도를 갱신
(Tegra 플랫폼에서는 BPMP Thermal Driver가 BPMP Firmware로부터 온도 정보를 조회)
- (선택) 온도 갱신 과정에서 Generic Netlink(gnl)를 통해 Thermal Event를 사용자 공간에 브로드캐스트
사용자 공간에서 libnl 등을 이용하여 이벤트 수신 가능
- 현재 온도와 Trip Point를 비교하여 활성화된 Trip 여부를 확인
- 활성화된 Trip이 존재하면 해당 Thermal Zone에 연결된 Governor를 호출
- Governor는 Cooling Device의 상태를 조정하여 Thermal Throttling을 수행
(별도 설정이 없으면 step_wise Governor가 기본으로 사용됨)

throttling

- Thermal Governor는 Cooling Device의 set_cur_state()를 호출하여 Cooling State를 조정
- Cooling Device는 상태 조회(get_cur_state)와 상태 변경(set_cur_state) 인터페이스를 구현하여 등록
- cpufreq cooling device의 경우 Cooling State에 따라 CPU 주파수 제한 수준을 조정

Throttle 수행 과정

cpufreq_set_cur_state()
↓
freq_qos_update_request()
↓
cpufreq policy 최대 주파수 제한
↓
cpufreq Driver 적용
↓
CPU 주파수 변경 적용

- Tegra 플랫폼에서는 최종적으로 cpufreq Driver가 클럭 설정을 변경하여 CPU 주파수를 낮춤
- Throttling 관련 이벤트(Trip Point, Cooling Device 상태 변경 등)는 Generic Netlink(genl)를 통해 사용자 공간에 전달 가능

```
/* Bind cpufreq callbacks to thermal cooling device ops */  
  
static struct thermal_cooling_device_ops cpufreq_cooling_ops = {  
    .get_max_state = cpufreq_get_max_state,  
    .get_cur_state = cpufreq_get_cur_state,  
    .set_cur_state = cpufreq_set_cur_state,  
};
```

테스트 전체 흐름

cpu-thermal zone의 emul_temp(sysfs)를 이용하여 Thermal Trip을 강제로 발생시키고
Thermal Governor와 CPUFreq Cooling Device를 통한 CPU 주파수 제한 동작을 검증

1. 현재 상태 확인
2. emul_temp를 이용한 Thermal Trip 발생
3. Cooling Device state 변화 확인
4. CPUFreq policy 제한 확인
5. 실제 CPU 주파수 감소 확인

테스트

1. 테스트 전 상태 확인

```

```
cat /sys/class/thermal/thermal_zone0/temp
cat /sys/class/thermal/thermal_zone0/cdev*/cur_state
cat /sys/devices/system/cpu/cpufreq/policy*/scaling_max_freq
cat /sys/devices/system/cpu/cpufreq/policy*/scaling_cur_freq
```

```

2. emul_temp로 trip 발생

cpu-sw-slowdown Trip Point(100°C)를 기준으로 emul_temp를 100000(m°C)으로 설정

```

```
echo 100000 > /sys/class/thermal/thermal_zone0/emul_temp
```

```

3. Cooling Device state 변화 확인

```

```
cat /sys/class/thermal/thermal_zone0/cdev*/cur_state
```

```

변화 확인

- cpufreq-cpu0 : 0 → 1
- cpufreq-cpu4 : 0 → 1

step_wise Governor는 Cooling State를 단계적으로 증가시키므로 cpufreq Cooling Device의 State가 0 → 1로 변경됨을 확인

테스트

4. CPUFreq Policy 제한 확인

```

```
cat /sys/devices/system/cpu/cpufreq/policy*/scaling_max_freq
```

```

before : 1510400

after : 1497600

Cooling Device State가 0 → 1로 증가하면서 CPUFreq Policy의 최대 주파수가 1510400 → 1497600으로 제한됨을 확인
본 테스트에서는 Cooling Device State가 1단계만 증가하여 주파수 변화 폭은 크지 않았음

sysfs 를 통해서 온도를 조금씩 계속 증가시키면 step wise governor 가 cooling device 의 상태를 1단계씩 증가시켜서
주파수 제한이 더 걸리는 것을 확인 가능

5. 실제 주파수 감소 확인

```

부하를 걸고: stress-ng --cpu 8

동시에 관측: watch -n 0.5 'cat /sys/devices/system/cpu/cpufreq/policy\*/scaling\_cur\_freq'

```

before : 1510400

after : 1497600

결과 해석

- DTS에 정의된 Thermal Zone, Trip Point, Cooling Map의 동작을 분석
- BPMP → Thermal Core → Governor → Cooling Device → CPUFreq 제어 흐름을 확인
- emul_temp를 이용한 검증을 통해 Thermal Trip 발생 시 CPUFreq Policy가 제한되고 실제 CPU 주파수가 감소함을 확인
- Linux Thermal Framework가 CPU 발열 제어를 위해 동작하는 과정을 분석

Siho Kim

Thank You